

Exercise 5: Connect & Extend

Session: 5 — MCP Servers & External Tools **Duration:** 30 minutes **Project:** Same BookStore API.

Goal: Build a custom SQLite MCP server that exposes the bookstore database to Claude. Then create a security-auditor subagent, and compare AI responses with and without each extension loaded.

Stuck on a question, or short on time? Say "*just tell me*" to Claude and it will skip ahead to a direct answer.

Tasks

1. Build and Configure the SQLite MCP Server (8 min)

The bookstore uses a SQLite database (`store.db`). Without an MCP, Claude cannot reach the database (this project's `CLAUDE.md` forbids any other route to `store.db`). It can only infer the schema and the data from the source code, and it cannot verify anything against real data. With MCP, it can access information about the database and knows to query the schema and run real SQL. The server lives at `mcp-sqlite/main.go` and exposes two tools:

- `get_table_definitions` — returns `CREATE TABLE` statements and column info
- `execute_query` — executes a read-only `SELECT` query and returns JSON rows

Step 1 — Make sure the bookstore database exists. From the project root, start the bookstore server once (it seeds the database on the first run):

```
cd bookstore-py && python3 main.py &
# wait ~2 seconds, then stop it – we just need store.db to exist
kill %1
cd ..
```

Verify the `store.db` file exists:

```
ls -lh bookstore-py/store.db
```

Step 2 — Compile the MCP server to a binary. Claude Code launches the server as a subprocess and talks to it over stdin/stdout, so it needs an executable:

```
cd mcp-sqlite && go build -o ../mcp-sqlite-server . && cd ..
```

This produces `mcp-sqlite-server` in the project root. Confirm it compiled:

```
./mcp-sqlite-server --help
```

You should see the `-db` flag printed. If so, the server is working.

Step 3 — Register the server with Claude Code (project scope). Use the absolute path so Claude Code can find the binary regardless of working directory, and point it at `bookstore-py/store.db`:

```
claude mcp add \  
  --transport stdio \  
  --scope project \  
  sqlite-bookstore \  
  -- $(realpath mcp-sqlite-server) -db $(realpath bookstore-py/store.db)
```

Step 4 — Verify the server is registered:

```
claude mcp list
```

You should see `sqlite-bookstore` in the list. Inside an active Claude Code session, type `/mcp` to confirm it shows as connected and lists both tools: `get_table_definitions` and `execute_query`.

Done when: `claude mcp list` shows `sqlite-bookstore`, and `/mcp` shows it connected with both tools listed.

2. Compare AI Responses: Without vs With MCP (5 min)

Round A — without MCP (disable it temporarily):

```
claude mcp remove sqlite-bookstore
```

Open a fresh Claude Code session. You are about to ask it this:

"How many books are in the bookstore database? Which author has the most books? Write a SQL query that returns all books with their author name and average rating, sorted by rating descending."

Before you send it, bet on how Claude will handle each of the three parts: the book count, the top author, and the SQL query. Claude has no database access yet, so it can only infer answers from the source code. Which answers will it hedge, and which could be wrong if the data has changed since seeding?

Now ask it.

Note three things: the book count, the top author, and one sentence on how sure Claude said it was. Short notes are enough. Also notice:

- Does it say that its answers are derived from code, not from the live database?
- Are the column names correct (`author_id` , `review_text` , `rating`)?
- Does it join the right tables?

Round B — with MCP (re-add it):

```
claude mcp add \  
  --transport stdio \  
  --scope project \  
  sqlite-bookstore \  
  -- $(realpath mcp-sqlite-server) -db $(realpath bookstore-py/store.db)
```

Open a **new** Claude Code session (so MCP connects on startup) and ask the **exact same question**.

Note the same three things for this round. Then observe the difference:

- Claude now calls `get_table_definitions` first — watch for the tool call in the output
- It then calls `execute_query` with a real query
- Column names are exact, joins are correct, results are real data

Ask a follow-up that would be impossible without live data:

"Which book has the highest average rating? Show me its title, author, and

the top 3 review texts."

Without MCP this is an inference from code. With MCP it is a fact.

Done when: your notes hold those three things for Round A and for Round B.

3. Create the Security-Auditor Subagent (9 min)

A subagent runs in its own isolated context window with its own tools and model. You will create one that specializes in OWASP security audits. The agent's instructions, model and effort are given. It uses a cheaper model (Haiku) to save cost. You write the two frontmatter fields that decide when it runs and what it may touch.

Step 1 — Create the agents directory:

```
mkdir -p bookstore-py/.claude/agents
```

Step 2 — Create `bookstore-py/.claude/agents/security-auditor.md` with this content. Leave `description` and `tools` empty for now:

```
---
name: security-auditor
description:
tools:
model: claude-haiku-4-5-20251001
effort: xhigh
color: red
---
```

You are a security engineer specializing in Python web applications and the OWASP Top 10. Your job is to find real vulnerabilities – not theoretical risks.

Scope

Audit only the files you are given. Do not modify any file.

Process

1. Run `Glob` with `bookstore/\*\*/\*.py` to find all source files
2. For each handler file in `bookstore/handler/`, read it fully
3. For each store file in `bookstore/store/`, read it fully
4. Check for the following vulnerabilities:

****A01 – Broken Access Control****

- Are there authorization checks on any endpoint?
- Can an unauthenticated user call DELETE or POST endpoints?

****A03 – Injection****

- Are SQL queries built with f-strings or `%` formatting instead of parameter binding?
- Are query parameters sanitized before use?

****A05 – Security Misconfiguration****

- Are error messages returned verbatim to HTTP clients?

- Does the server expose stack traces or internal paths?

****A07 – Identification and Authentication Failures****

- Is there any authentication middleware at all?

****A09 – Security Logging and Monitoring Failures****

- Are failed requests or suspicious inputs logged?

Output format

Write a report with this structure:

Security Audit Report

****Audited by**:** security-auditor

****Files reviewed**:** list every file you read

For each finding:

****[SEVERITY] OWASP Category – Short title****

- File: `path/to/file.py`, line N
- Description: what the vulnerability is
- Evidence: paste the relevant code snippet
- Recommendation: one concrete fix

Severity levels: CRITICAL, HIGH, MEDIUM, LOW, INFO

End with a ****Summary**** table: | Severity | Count |

Return the report in exactly this structure, starting with the
`### Security Audit Report` heading. The main session saves your reply to
a
file as it is. Add no text before or after the report.

Step 3 — Fill in the two empty fields yourself:

- `description` : the trigger. Claude reads only this text to decide whether to delegate. Say what the agent checks. Then say when Claude should invoke it: what a user might ask, and which code change should start it. Improve this weak version:

```
description: Reviews code for security issues.
```

- `tools` : a comma-separated list of tool names. This agent must never modify code. Give it the smallest set that still lets it find and read source files.

Step 4 — Run `/mcp-coach 3`. The coach reads your file and grades the description as a trigger. It asks questions, it does not write the text for you. Revise until it gives the greenlight.

Step 5 — Test whether the subagent auto-triggers. Open a **new** Claude Code session in `bookstore-py` (agents load on startup). Ask, without naming `security-auditor`:

```
"Do a security audit of the bookstore API. Save the full report, exactly as it was produced, to docs/security-audit.md."
```

Watch the tool calls in the output. If your description works, Claude delegates to `security-auditor` automatically. The audit runs in a separate context, so your main conversation stays clean. The subagent cannot write files, so it returns the report and the main session saves it. The report must start with the `**Audited by**:` `security-auditor` line. That line shows that the subagent wrote it, not the main session.

If Claude did the audit itself, your trigger did not fire. Improve the description and try again in a new session.

Done when: `bookstore-py/.claude/agents/security-auditor.md` has both fields filled in, and `bookstore-py/docs/security-audit.md` holds the report.

Closing round (6 min)

Think about these. One keyword per question is enough. The trainer may ask students to answer out loud.

1. **MCP schema awareness:** did Claude call `get_table_definitions` before every query, or only once? What does that tell you about how Claude reuses tool results within a session?
2. **Subagent delegation:** did Claude delegate automatically, or did you have to trigger it explicitly? What would you change in the `description:` field to make auto-delegation more reliable?
3. **Real-world applications:** which MCP servers would save your team the most time, and what would you need to check before connecting one?
4. **Cost vs capability:** the security auditor uses Haiku to save cost. Name one task in your own workflow that a cheaper model could handle. Say why it would not lose quality.

Then start `/verify-exercise 5` in the same project folder. It checks everything by itself: your MCP registration, a live call to the MCP server, your subagent file, and the saved report. It asks you one question only. Bring its report to the closing round.

Done when: you have successfully run `/verify-exercise 5`, and you have an answer ready for all four questions above.